

AI Voice Calling Automation System

Kotlin Frontend + Python + Node.js + MongoDB Architecture

Based on your requirements, this documentation is designed for:

- SIM-based real phone calling
- Excel lead import
- Fixed question asking
- Caller voice response capture
- Speech-to-text transcription
- MongoDB storage
- Fully open-source stack
- No paid APIs
- Kotlin frontend
- Python AI/audio engine
- Node.js orchestration backend

1. Project Overview

Project Name

AI Voice Calling Automation System

Primary Objective

System automatically:

1. Reads phone numbers from Excel
2. Calls users using Android SIM
3. Plays fixed WAV questions
4. Captures caller responses
5. Converts speech → text
6. Stores responses in MongoDB
7. Shows live dashboard updates

2. Final Technology Stack

Layer	Technology
-------	------------

Frontend Dashboard	Kotlin (Compose Multiplatform / Ktor Client)
Backend API Gateway	Node.js + Express
AI & Audio Engine	Python + FastAPI
Database	MongoDB
Android Automation	ADB + uiautomator2
Speech Recognition	Faster-Whisper
Real-time Updates	Socket.IO
Excel Processing	pandas + openpyxl
Audio Processing	pydub + sounddevice
Queue System	BullMQ / Redis
Deployment	Docker

3. Why Hybrid Architecture?

Node.js Responsibilities

Node.js handles:

- Campaign management
- REST APIs
- WebSocket
- Queue system
- Dashboard communication
- Authentication
- MongoDB operations
- Multi-device orchestration

Python Responsibilities

Python handles:

- ADB operations

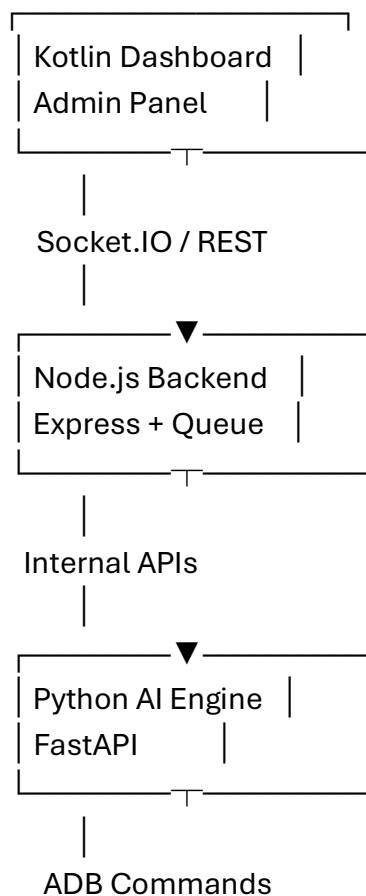
- Audio playback
- Call handling
- Speech-to-text
- Recording processing
- AI workflows
- Whisper inference

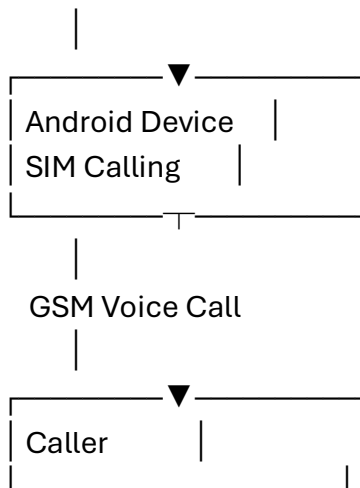
Kotlin Responsibilities

Kotlin frontend handles:

- Admin dashboard
- Real-time call monitoring
- Analytics
- Campaign control
- Device status
- Logs visualization

4. Full System Architecture





5. Real Calling Flow

Excel Import

↓

Node.js Queue

↓

Python Call Worker

↓

ADB Initiates Call

↓

Call Connected

↓

Greeting.wav Play

↓

Question 1.wav

↓

Caller Response Recording

↓

Whisper Transcription

↓

MongoDB Save

↓

Question 2.wav

↓

Repeat

↓

Call End

6. Project Folder Structure

ai-call-system/

```
├── frontend-kotlin/
│   ├── src/
│   ├── ui/
│   ├── screens/
│   ├── websocket/
│   ├── api/
│   └── build.gradle.kts
├── backend-node/
│   ├── src/
│   │   ├── api/
│   │   ├── controllers/
│   │   ├── routes/
│   │   ├── services/
│   │   ├── queues/
│   │   ├── socket/
│   │   ├── mongodb/
│   │   └── utils/
│   ├── package.json
│   └── .env
├── ai-python/
│   ├── main.py
│   ├── adb/
│   ├── whisper/
│   ├── audio/
│   ├── recorder/
│   ├── conversation/
│   ├── transcription/
│   └── workers/
```

```
|   └─ requirements.txt
|
|   └─ audio/
|       ├── greeting.wav
|       ├── question_1.wav
|       ├── question_2.wav
|       └─ closing.wav
|
|   └─ leads/
|       └─ leads.xlsx
|
|   └─ recordings/
|
|   └─ logs/
|
|   └─ docker/
|
|   └─ README.md
```

7. Hardware Requirements

Hardware	Recommended
RAM	16GB
CPU	Ryzen 7 / Intel i7
GPU	RTX 3060 Optional
Android Phone	Android 10+
Storage	SSD Recommended

8. Android Requirements

Enable Developer Options

Mandatory:

- USB Debugging

- Stay Awake
- Allow ADB Permissions

9. Required Software Installation

Install Python

```
sudo apt install python3 python3-pip
```

Install Node.js

```
sudo apt install nodejs npm
```

Install MongoDB

```
sudo apt install mongodb
```

Install ADB

```
sudo apt install android-tools-adb
```

10. Verify Device Connection

```
adb devices
```

Expected:

List of devices attached

ZX1G22XXXXX device

11. Python Dependencies

```
pip install fastapi uvicorn
```

```
pip install pandas openpyxl
```

```
pip install pymongo
```

```
pip install faster-whisper
```

```
pip install uiautomator2 adbutils
```

```
pip install pydub playsound sounddevice
```

```
pip install python-socketio
pip install numpy scipy
```

12. Node.js Dependencies

```
npm install express mongoose socket.io cors dotenv
npm install bullmq redis
npm install multer xlsx
npm install axios
npm install nodemon
```

13. Excel Format

leads.xlsx

Name	Phone
Rahul	9876543210
Amit	9988776655

14. Excel Reader Module

Python Example

```
import pandas as pd

df = pd.read_excel("leads/leads.xlsx")

numbers = df["Phone"].tolist()

print(numbers)
```


15. MongoDB Schema

```
{
  "phone_number": "9876543210",
  "call_status": "completed",
  "started_at": "2026-06-03T17:00:00",

  "conversation": [
    {
      "question_id": 1,
      "question": "Need website service?",
      "answer": "haan ecommerce website chahiye"
    },

    {
      "question_id": 2,
      "question": "Budget?",
      "answer": "50 thousand"
    }
  ]
}
```

16. ADB Calling Module

Python Call Example

```
import subprocess
```

```
number = "9876543210"
```

```
subprocess.run([
    "adb",
    "shell",
    "am",
    "start",
    "-a",
    "android.intent.action.CALL",
    "-d",
    f"tel:{number}"
])
```

])

17. Real Audio Strategy

Important Android Limitation

Android 10+ blocks:

- Internal telecom audio capture
- Direct call audio injection

Therefore:

Recommended Stable Method

PC WAV Playback

↓

Phone Speaker Plays Audio

↓

Phone Mic Captures Audio

↓

Caller Hears Questions

This is the safest non-rooted solution.

18. Question Configuration

```
QUESTIONS = [  
  {  
    "id": 1,  
    "audio": "audio/question_1.wav",  
    "text": "Do you need website service?"  
  },  
  
  {  
    "id": 2,  
    "audio": "audio/question_2.wav",  
    "text": "What is your budget?"  
  }  
]
```

19. Audio Playback Module

```
from pydub import AudioSegment
from pydub.playback import play

audio = AudioSegment.from_wav("audio/question_1.wav")

play(audio)
```

20. Recording Module

```
import sounddevice as sd
from scipy.io.wavfile import write

fs = 44100

seconds = 7

recording = sd.rec(
    int(seconds * fs),
    samplerate=fs,
    channels=1
)

sd.wait()

write("recordings/output.wav", fs, recording)
```

21. Speech-To-Text Module

```
from faster_whisper import WhisperModel

model = WhisperModel("base")

segments, info = model.transcribe(
    "recordings/output.wav"
```

```
)  
  
full_text = ""  
  
for segment in segments:  
    full_text += segment.text  
  
print(full_text)
```

22. Call State Flow

PENDING
↓
CALLING
↓
RINGING
↓
CONNECTED
↓
PLAYING_AUDIO
↓
LISTENING
↓
TRANSCRIBING
↓
COMPLETED

23. Node.js Queue Architecture

Why Queue Needed?

Without queue:

- Device overload
- Call conflicts
- Random failures

Queue manages:

- Sequential calls

- Retry system
- Multi-device scaling
- Failure recovery

24. BullMQ Queue Example

```
import Queue from "bull";

const callQueue = new Queue("call-queue");

callQueue.add({
  phone: "9876543210"
});
```

25. FastAPI Service Example

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {
        "status": "running"
    }
```

26. Node.js API Endpoints

Endpoint	Purpose
/upload-leads	Upload Excel
/start-campaign	Start Calls
/stop-campaign	Stop Campaign
/call-status	Live Status

/analytics	Reports
/devices	Device Status

27. Kotlin Frontend Features

Dashboard Modules

- Campaign Management
- Upload Excel
- Start/Stop Calling
- Live Call Monitoring
- Analytics
- Device Monitoring
- Logs Viewer
- Real-time Status

28. Kotlin Frontend Recommended Stack

Component	Technology
UI	Compose Multiplatform
HTTP Client	Ktor
WebSocket	Socket.IO Client
State Management	ViewModel
Charts	MPAndroidChart
Navigation	Voyager

29. Real-Time Socket Events

Events

call_started
call_connected
question_played
recording_started

transcription_done
call_completed
call_failed

30. Security Considerations

Required

- JWT Authentication
- Environment Variables
- MongoDB Authentication
- API Rate Limiting
- Input Validation
- Device Access Restrictions

31. Error Handling Strategy

Failure	Solution
Device disconnected	Retry ADB
Call rejected	Skip
No response	Retry question
Whisper failure	Save raw recording
MongoDB failure	Queue retry
Audio playback failure	Restart audio engine

32. Multi-Device Scaling

Future scaling architecture:

Node.js Queue

↓

Python Worker 1 → Android 1

Python Worker 2 → Android 2

33. Recommended Development Phases

Phase 1 — MVP

Features:

- Excel Import
- Auto Calling
- WAV Playback
- Auto Hangup

Estimated: 2–3 Days

Phase 2 — Response Capture

Features:

- Recording
- Whisper STT
- MongoDB Storage

Estimated: 5–7 Days

Phase 3 — Dashboard

Features:

- Kotlin Frontend
- Real-time Monitoring
- Analytics

Estimated: 1–2 Weeks

Phase 4 — Production Scaling

Features:

- Multi-device support
- Queue management
- Docker deployment
- Auto retries

Estimated: 2–4 Weeks

34. Future Enhancements

Planned Features

- AI dynamic conversations
- GPT-based replies
- Voice cloning
- CRM integration
- Sentiment analysis
- Multi-language calling
- Semantic search
- Human handoff
- SIP calling support

35. Final Recommended Stack

Kotlin Frontend

+

Node.js Backend

+

Python AI Engine

+

ADB Android Automation

+

MongoDB

+

Faster Whisper

+

Socket.IO

36. Final Notes

This architecture is:

- Fully open-source
- Offline capable

- Self-hosted
- No recurring API cost
- Production scalable
- Multi-device extensible

Recommended strategy:

Start with:

- Fixed WAV questions
- Sequential calls
- Single Android device

Then gradually move toward:

- Real-time AI conversations
- Parallel calling
- Multi-device orchestration
- Production scaling